



모던 자바 최적화 & 마이그레이션 플레이북

Java 8 리팩터링부터 Java 21 마이그레이션까지, 읽기 쉽고 빠른 코드를 위한 실전 가이드



가독성 & 디자인

- 람다(Lambdas)
- 객체지향 패턴의 함수형 전환
- `Optional` 안티패턴



동시성 & 성능

- `CompletableFuture` 비동기 제어
- 스트림 병렬화 조건
- GC 튜닝 & 메모리 구조



마이그레이션

- Java 17/21 LTS 전환
- 가상 스레드 (Virtual Threads)
- `InaccessibleObjectException` 해결

Design Pattern Transformation Matrix

	Traditional OOP	Modern Java (Lambda)
전략 패턴 (Strategy)	인터페이스 구현체(<code>ConcreteStrategy</code>) 생성	<code>Predicate<T></code> / 커스텀 람다 표현식 직접 전달
템플릿 메서드 패턴 (Template Method)	추상 클래스 상속 및 <code>Primitive</code> 메서드 오버라이드	<code>Consumer<T></code> 파라미터화로 상속 구조 제거
의무 체인 패턴 (Chain of Responsibility)	작업처리 추상 클래스 상속 및 연결	<code>UnaryOperator<T></code> 의 <code>andThen()</code> 메서드 조합
팩토리 패턴 (Factory)	분리된 <code>Factory</code> 클래스 및 복잡한 생성 로직	<code>Supplier<T></code> 를 활용한 지연 생성(Lazy Instantiation)

Anti-Pattern Radar

Anti-Patterns ❌

- `Optional` 변수에 `null` 할당
- 값 확인 없이 무작정 `.get()` 호출
- 필드 타입이나 컬렉션 원소(
`List<<Optional<<T>>>>`)로 사용

Best Practices ○

- 빈 값 초기화는 `Optional.empty()` 사용
- `orElse()`, `orElseGet()`,
`orElseThrow()`로 안전한 언래핑
- 메서드의 반환 타입(Return Type)으로만
제한적 사용
- 기본형은 `OptionalInt`, `OptionalLong`
등 전용 클래스 사용

💡 `Optional.equals`가 구현되어 있으므로, 값 비교 시 `.get()` 호출 없이 직접 비교 가능.

Asynchronous Paradigm Comparison

기능 구분	Future	CompletableFuture
결과 조회 (Blocking)	<code>.get()</code> 호출 시 스레드 블로킹 (동기적 대기)	논블로킹 콜백 (<code>thenApply</code> , <code>thenAccept</code>)
작업 체이닝 (Chaining)	직접 구현 어려움	메서드 체이닝 지원 (선언적 파이프라인)
조합 (Composition)	다중 Future 수집 시 복잡한 동기화 필요	<code>thenCompose</code> (순차), <code>thenCombine</code> (병렬), <code>allOf</code> (전체)
예외 처리 (Exception)	<code>try-catch</code> 및 <code>ExecutionException</code> 언래핑 필수	<code>handle()</code> (성공/실패), <code>exceptionally()</code> (예외 전용)

CompletableFuture Asynchronous Pipeline



💡 실무 성능 개선 사례: 1000개 비디오 처리 기준, 싱글스레드 순차 처리(50초) 대비
CompletableFuture + `allOf()` 병렬 처리 시 8초로 약 6배 이상의 성능 향상.

스트림 병렬화(.parallel()) 적용 판단 트리

데이터 크기가 병렬화 오버헤드를 상쇄할 만큼 충분히 큰가?

.sequential() 사용

람다 표현식에 상태를 변경하는 사이드 이펙트가 있는가?

동기화 병목 우려, 사용 금지

소스가 순서를 보장하며(예: ArrayList), 순서 의존적 연산 distinct())이 포함되어 있는가?

결과 순서가 무관하다면 .unordered().parallel()을 사용하여 병렬 성능 극대화 (예: HashSet 수집 시)

안전하게 .parallel() 적용 가능

💡 실증 데이터: 64만 라인 코드 분석 결과, 안전한 비순차 변환 적용 시 평균 1.55배 속도 향상.

Refactoring Preview: Parallel Stream Optimization

Original Source

```
22
23 // sort widgets by weight.
24 List<Widget> sortedWidgets =
    widgets.stream().sorted
      (Comparator.comparingInt
        (Widget::getWeight))
25
26 .collect(Collectors.toList())
27
28 // an "ordered" collection of
29 Collection<Widget> orderedWidgets
    = new ArrayList<>();
30 // populate the collection ..
```

→ Refactored Source

```
22
23 // sort widgets by weight.
24 List<Widget> sortedWidgets =
    widgets.unordered().parallel
      .sorted(Comparator.compar
        (Widget::getWeight))
25
26 .collect(Collectors.toSet())
27
28 // an "ordered" collection of
29 Collection<Widget> orderedWidgets
    = new ArrayList<>();
30 // populate the collection ..
```

?

< Back

Next >

Cancel

Apply

JVM Memory Model Evolution: Java 7 vs. Java 8+

Java 7 Architecture



클래스 로드/언로드가 잦은 프레임워크(Spring, Hibernate) 사용 시 메모리 조각화 및 OutOfMemoryError(OOM) 발생 위험 높음.

Java 8+ Architecture



OS가 네이티브 메모리를 사용하여 동적 할당.
GC 부담 감소 및 메모리 부족(OOM) 문제 대폭 개선.

GC Tuning Diagnostic Board

G1 GC (대부분의 Spring Boot 기본값)

Profile: 일반적인 서버 애플리케이션, 예측 가능한 STW(Stop-the-World) 요구 환경.

```
-XX:+UseG1GC  
-XX:MaxGCPauseMillis=200  
-XX:InitiatingHeapOccupancyPercent=45  
-XX:+UseStringDeduplication
```

ZGC

Profile: 대규모 데이터 처리 (TB 단위), 마이크로초 단위의 초저지연 요구 환경.

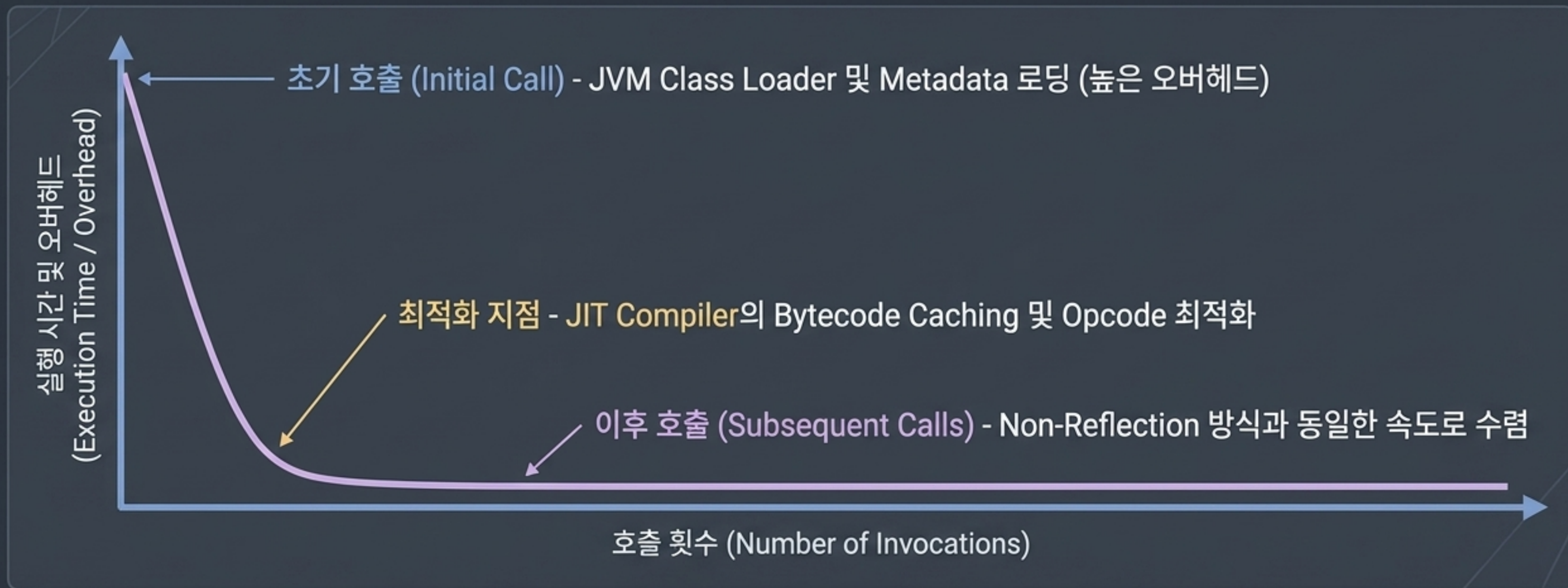
```
-XX:+UseZGC
```

Shenandoah GC

Profile: 빠른 응답 속도가 최우선인 환경, 가비지 수집과 애플리케이션 스레드 동시 실행.

```
-XX:+UseShenandoahGC  
-XX:ShenandoahGCHeuristics=adaptive
```

Reflection API Performance Myth vs Reality



결론: Reflection API는 초기 호출 이후 캐싱을 통해 성능 격차가 사라집니다.
런타임 유연성이 필요한 프레임워크 설계 시 적극 도입 가능합니다.

Java LTS Version Comparison

Java 17 LTS

목표: 보수적 업그레이드 및 레거시 시스템의 1차 안정화

핵심 이슈 (JEP 403): JDK 내부 API 강한 캡슐화 적용

주의사항: 리플렉션으로 내부 API 접근 시 `InaccessibleObjectException` 발생.

임시 우회 방안: `--add-opens` JVM 플래그 사용

Java 21 LTS

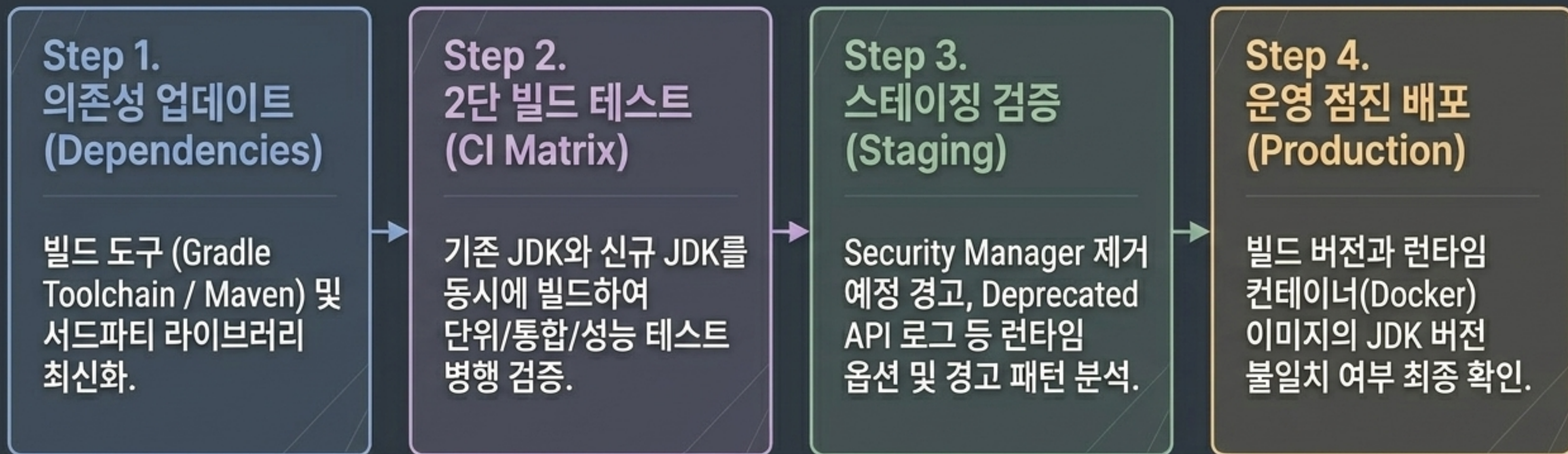
목표: 신규 프로젝트 및 대량의 동시 요청 처리 서버

핵심 기능: 가상 스레드 (Virtual Threads) 도입

효과: I/O 중심(DB/외부 API 대기) 서버에서 기존 스레드 풀 튜닝 없이 동시성 한계 돌파

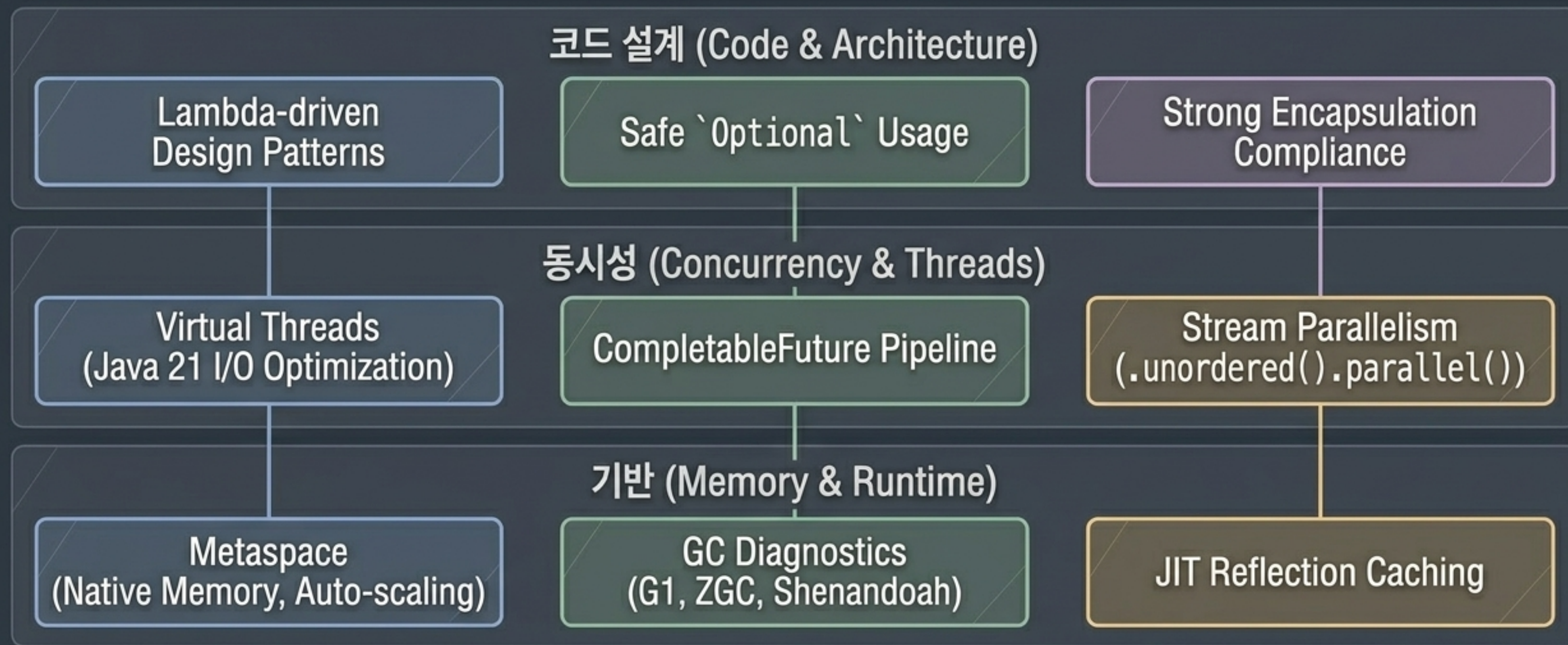
추가 기능: `Sequenced Collections` 도입으로 직관적인 컬렉션 제어 가능

CI/CD Pipeline Modernization Strategy



🔴 **Top Error to Watch:** `InaccessibleObjectException` 발생 시 바이트코드 조작 라이브러리 및 APM 에이전트 버전 즉시 확인 및 업그레이드.

Modern Java Ecosystem Map



안전한 마이그레이션과 현대적 문법의 결합 — 유연하고 확장 가능한 엔터프라이즈 자바의 완성.